BRAC University	**Final Examination**
Department of Computer Science and Engineering	**Set: B**
Course Code: CSE321	**Duration: 2 Hours**
Course Title: Operating Systems	**Total Marks: 35**

Name: ______________________________

ID: ______________________ **Section:** ______________

Instructions:

- Answer all questions.
- Marks are shown in brackets, and difficulty is shown as a tag (e.g., CA-E, CA-M, CA-D).
- Understanding the question is part of the exam.
- Write the answers for Questions 1(a), 1(b), 1(e), 2(a), 3(a), 3(b), and 3(c) in the designated answer boxes in this question paper.
- Write the answers for Questions 1(c), 1(d), 2(b), 2(c), 2(d), 2(e), and 2(f) in the answer script.
- For conceptual questions, write only the precise and relevant answer in short. Do not write detailed or descriptive explanations.

1. CO2 File Systems **[12]**

a) Nusyba is trying to create a text file named "FinalExam". The path of the newly created file is "/CSE321/Fall2025/Questions/FinalExam.txt".
Mention in which parts of the file system the write operations are done? **CA-E [2]**
Answer:

b) Analyze the journal entry given below. **CA-M [2]**

TxB id=50	**I(F7) ptr: 111**	**D(F7) address : 111**	**TxE id=50**	**TxB id=51**	**I(F8) ptr: 121**	**D(F8) address : 121**	**TxE id=51**	**TxB id=52**	**I(F9) ptr: 121**	**TxE id=52**

(i) Identify what type of issue will occur during the recovery after the system crashes? [1]
(ii) Mention how the file system can prevent it from happening. [1]
Answer:

I	
II	

c) A file system uses a UNIX inode data structure that contains 6 direct block addresses, 1 single indirect block, 2 double indirect blocks, and 1 triple indirect block. The size of each block is 32 bytes, and the size of each block address is 4 bytes. Assume a reader is trying to read the 240th byte of the data from this file.
Find out how many disk read operations are needed to access that Byte? **CA-D** [**3**]

d) A file system has an inode size = 128 Bytes and a block size = 1 KB. The first 7 blocks contain the superblock, journal, group descriptor, journal checksums, orphan list, data bitmap, and inode bitmap. **CA-E** [**3**]

- **(i)** **Calculate** the address of the inode number 27. [**1**]
- **(ii)** The disk is sector addressable, and the size of each sector is 1 KB. **Calculate** the sector address of the inode block. [**2**]

e) A file system creates and updates files very frequently. **Write** what will be the impact on file system performance if the space allocated for journal blocks is small? **CA-D** [**2**]
Answer:

2. CO3 Memory Management [18]

a) **Explain** whether internal fragmentation can be fully solved using paging. If yes, why? If not, how can its severity be reduced? **CA-E** [**2**]
Answer:

b) Consider an operating system with a 48-bit logical address, and a page size of 128KB, running on 4GB RAM. The page table for each process is indexed with the page number, and contains the frame index and three permission bits. Assuming a process uses up its maximum available memory in the operating system, **calculate** the size of the page table of the process. **CA-M** [**3**]

c) Process A fork Process B and both execute the following code snippet. Assume that the array arr is initially not loaded in RAM. The page size is 2KB.
Calculate what will be the i) maximum and ii) minimum number of pages newly created after the completion of the code snippet? **CA-D** [**1.5+1.5=3**]

```
for (int i = 0; i < 1400; i++) {
    arr[i] = arr[i] * 2
}
```

d) Consider a system with TLB hit ratio of 95.35%, and the TLB access time of 24 ns. If Effective Access Time is 390.275 ns, then **calculate** the main memory access time. **CA-E** [**3**]

e) The OS is attempting to create a process that utilizes a single-level paging system. The current Free Frame List has no two consecutive free frames in the RAM, but there are numerous free frames available. The page size is 1KB. And the logical address space is 20 bits. **Write** can the process be created? Provide necessary mathematical calculations. **CA-D** [3]

f) A process runs in a system with 3-level paging, and it has a logical address space of 16 bits. In the system, the page size is 64 Bytes, the size of each entry in the page table is 4 Bytes, and the size of main memory is 2 KB.
After paging, the logical address space of the process looks like: **CA-M** [2*2=4]

2nd outer pg#	**outer pg#**	**inner pg#**	**offset**
P1	**P2**	**P3**	**d**
2	**4**	**4**	**6**

Necessary page table information is given below:

2nd outer PMT		
Page #	**Frame #**	**V/I bit**
0	**50**	**v**
1		
2	**23**	**v**
3		

outer PMT						**inner PMT**					
Frame 23			**Frame 50**			**Frame 7**			**Frame 2**		
Pg#	**Fr#**	**V/I bit**	**Pg#**	**Fr#**	**V/I bit**	**Pg#**	**Fr#**	**V/I bit**	**Pg#**	**Fr#**	**V/I bit**
0			**0**			**0**			**0**		
1			**1**			**1**			**1**		
2			**2**	**78**	**v**	**2**			**2**		
3			**3**			**3**	**19**	**v**	**3**	**6**	**v**
4			**4**			**4**			**4**		
5	**24**	**v**	**5**			**5**			**5**		
6			**6**			**6**			**6**		
7			**7**			**7**			**7**		
8			**8**			**8**			**8**		
9			**9**			**9**			**9**		
10			**10**	**2**	**v**	**10**			**10**		
11			**11**			**11**			**11**		
12			**12**			**12**	**45**	**v**	**12**	**95**	**v**
13			**13**			**13**			**13**		
14	**7**	**v**	**14**			**14**			**14**		
15			**15**			**15**			**15**		

In this system, if the CPU generates logical addresses i) 11059 and ii) 47321, then **map** the corresponding physical addresses of these logical addresses. If the physical address is not valid, then write **"invalid"**.

3. CO1 Protection and Security [5]

a) Your OS enforces that whenever a domain is created, it is static and its contents cannot be changed. But you have to add some additional objects to your access list for a specific task after the domain has been initialized for it.
Identify what can be the possible solution to this problem? **CA-M** [2]
Answer:

b) Consider the code snippet given below: **CA-M [2]**

```
1  #include <stdlib.h>
2  #include <string.h>
3
4  struct log_entry {
5      char header[8];
6      char *msg;
7  };
8
9  void log_event(char *title, char *msg) {
10     struct log_entry *entry = malloc(sizeof(log_entry));
11     entry->msg = malloc(128);
12
13     strcpy(entry->title, title);
14     strncpy(entry->msg, msg, 128);
15
16     add_to_log(entry);
17 }
```

(i) Identify Which line of the code is responsible for the memory safety vulnerability? [**1**]

(ii) Justify the reason behind this. [**1**]

Answer:

I	
II	

c) A system requires users to authenticate before performing transactions. An attacker records a valid authentication message sent earlier and later resends the same message to the server to gain access, without knowing the user's credentials.

Identify the security violation method used in this scenario. **CA-E [1]**

Answer:
